

n8n Workflow Build Guide — Node by Node

Use this guide if `workflow.json` does not import cleanly into your n8n version. Build the workflow manually by adding nodes in the order below.

Prerequisites

- FastAPI server running locally on port 8888
 - ngrok tunnel running: `ngrok http 8888` → copy the HTTPS URL as `NGROK_URL`
 - Groq API key (free) — get one at <https://console.groq.com> (starts with `gsk_`)
-

Node 1 — Form Trigger

Add: Click + on the canvas → search "Form" → select **n8n Form Trigger**

Configure:

- Form Title: Social Media Post Request
- Form Description: Enter a post topic and scheduled publish time.
- Add Field 1:
 - Label: `topic`
 - Type: Text
 - Required: Yes
- Add Field 2:
 - Label: `scheduled_at`
 - Type: Text
 - Required: Yes
- Response Mode: Last Node

Why: This is the workflow's entry point. Submitting the form triggers the entire automation.

Node 2 — Generate Post Content (Code node)

Add: Click + → search "Code" → select **Code**

Configure:

- Language: JavaScript
- Paste the entire contents of the `jsCode` field from `workflow.json` node `node-2-llm-retry`

The code does three things:

1. Reads `topic` and `scheduled_at` from the form input
2. Calls Groq (Llama 3.3 70B) up to 5 times, checking word count (90–150 words) after each call; throws and stops if it still can't comply
3. On each retry, sends a corrective prompt: "Your draft was N words, rewrite to 90-150"

Setup required: Open the Code node, find the line `const apiKey = 'PASTE_YOUR_GROQ_API_KEY_HERE'`; and replace the placeholder with your real key. Get a free key at <https://console.groq.com> → API Keys. (Settings → Environment Variables is Enterprise-only on n8n Cloud free tier, so the key is pasted directly into the node.)

Connect: Form Trigger → Generate Post Content

Node 3 — Compose Brand Image (HTTP Request)

Add: Click + → search "HTTP Request" → select **HTTP Request**

Configure:

- Method: POST
- URL: `YOUR_NGROK_URL/compose-image`
- Body Content Type: JSON
- Body (key-value):
 - `title = {{ $json.topic }}`

Why: FastAPI composites the topic text onto a fixed 1080×1080 brand card using Pillow. Doing this in Python (not n8n) gives precise pixel control. Returns `{ image_url, image_path }`.

Connect: Generate Post Content → Compose Brand Image

Node 4 — Request Human Approval (HTTP Request)

Add: Click + → HTTP Request

Configure:

- Method: POST
- URL: `YOUR_NGROK_URL/request-approval`
- Body Content Type: JSON
- Body (key-value):
 - `copy = {{ $('Generate Post Content').first().json.copy }}`
 - `image_url = {{ $json.image_url }}`

- `resume_webhook_url = {{ $execution.resumeUrl }}`

The critical field is `resume_webhook_url`: `$execution.resumeUrl` is n8n's auto-generated webhook URL for this execution. When the human clicks a button on the review dashboard, FastAPI POSTs to this URL to wake up the paused workflow.

Connect: Compose Brand Image → Request Human Approval

Node 5 — Wait for Human Decision (Wait node)

Add: Click + → search "Wait" → select **Wait**

Configure:

- Resume: On `webhook call` (not "After time interval")
- HTTP Method: POST (must match how FastAPI sends the callback — the default GET will reject it)

n8n generates the resume URL automatically and it's available via `$execution.resumeUrl` in the previous node.

What happens: When this node runs, the workflow execution is fully suspended. It only resumes when FastAPI POSTs the human's decision to `$execution.resumeUrl`. The POST payload is nested under `$json.body` in the next node (e.g. `$json.body.decision`, `$json.body.final_copy`).

Connect: Request Human Approval → Wait for Human Decision

Node 6 — Was Approved? (IF node)

Add: Click + → search "IF" → select **IF**

Configure:

- Add Condition:
 - Left value: `{{ $json.body.decision }}`
 - Operator: `is equal to`
 - Right value: `approved`

Why: The Wait node resumes with the payload `{ decision, final_copy, review_id }` nested under `body`. This IF checks `$json.body.decision` and routes to the publish path (true) or the rejection path (false).

Connect: Wait for Human Decision → Was Approved?

Node 7 — Publish to Facebook (HTTP Request)

Add: Click + → HTTP Request

Configure:

- Method: POST
- URL: YOUR_NGROK_URL/mock/facebook/post
- Body Content Type: JSON
- Body (key-value):
 - `image_url = {{ 'YOUR_NGROK_URL' + $('Compose Brand Image').first().json.image_url }}`
 - `description = {{ $json.body.final_copy || $('Generate Post Content').first().json.copy }}`
 - `scheduled_at = {{ $('Generate Post Content').first().json.scheduled_at }}`

Note: `$json.body.final_copy` is the approved copy (may be human-edited); the `||` fallback uses the original generated copy if the edited field is empty. `$( 'Compose Brand Image' ).first().json.image_url` reaches back to the image URL from node 3.

Replace `REPLACE_WITH_NGROK_URL` in **both** the URL field and the `image_url` body value.

Connect: Was Approved? (TRUE output, top port) → Publish to Facebook

Node 8 — Publish to Instagram (HTTP Request)

Add: Click + → HTTP Request

Configure:

- Method: POST
- URL: YOUR_NGROK_URL/mock/instagram/post
- Body Content Type: JSON
- Body (key-value):
 - `image_url = {{ 'YOUR_NGROK_URL' + $('Compose Brand Image').first().json.image_url }}`
 - `description = {{ $('Was Approved?').first().json.body.final_copy || $('Generate Post Content').first().json.copy }}`
 - `scheduled_at = {{ $('Generate Post Content').first().json.scheduled_at }}`

Connect: Publish to Facebook → Publish to Instagram

Node 9 — Rejected — Log and Stop (Set node)

Add: Click + → search "Set" → select **Set** (Edit Fields)

Configure:

- Add Field: status = rejected (String)
- Add Field: message = Human reviewer rejected the post. No publish action taken. (String)
- Add Field: rejected_copy = {{ \$json.body.final_copy }} (String)

Connect: Was Approved? (FALSE output, bottom port) → Rejected — Log and Stop

Final steps

1. **Save** the workflow (Ctrl+S)
2. **Activate** the workflow using the toggle in the top-right corner
3. Copy the Form Trigger's **Production URL**
4. Open it in a browser to test

See docs/n8n_setup.md for a full walkthrough including how to run a test end to end.